Add `pop_value` methods to container adaptors

Document #:	P3182R0
Date:	2024-04-16
Project:	Programming Language C++
Audience:	LEWG

Reply-to:	Brian Bi <bbi10@bloomberg.net>

Contents

1 Abstract

2 Introduction

3 Reasons for the status quo

- 3.1 Efficiency
- 3.2 Exception safety

4 Solution

- 4.1 `std::stack` and `std::queue`
  - 4.1.1 Throwing `pop_back` operations
  - 4.1.2 `move` versus `move_if_noexcept`
- 4.2 `std::priority_queue`

5 Alternative names

6 Wording

7 References

§ 1 Abstract

The `pop` methods of `std::stack`, `std::queue`, and `std::priority_queue` do not return the value popped, which is a longstanding pain point for users of the Standard Library. I show how to implement `pop_value` for `std::stack` and `std::queue` so that objects are not dropped even when an exception occurs, and propose the inclusion of these methods in the Standard Library. On the other hand, failure of a move operation will usually result in unrecoverable loss of data from `std::priority_queue` even under the status quo; therefore, I also propose the addition of `std::priority_queue::pop_value`.

§ 2 Introduction

The `pop` method appears in three container adaptors in the C++ Standard Library: namely, `std::stack`, `std::queue`, and `std::priority_queue`. The return type for each `pop` method is `void`; the popped object is destroyed and its value is not returned to the caller. Should the programmer wish to obtain the popped value, they must call the `top` or `front` method to obtain a reference to the object that is about to be popped, move the value of that object into some other object, and then call `pop`. Thus, idiomatic C++ requires three statements to accomplish what could be done with one if `pop` were to actually return the popped value:

```
class Widget {
    std::stack<int>      d_stack;
    std::array<int, 100> d_array;

    // ...

    int foo() {
       // invalid:
       // return d_array[d_stack.pop()];

       const auto top = d_stack.top();
       d_stack.pop();
       return top;
    }
}
```

The fact that `pop` does not return the popped value is a source of widespread annoyance among C++ users.

§ 3 Reasons for the status quo

§ 3.1 Efficiency

The `std::stack`, `std::queue`, and `std::priority_queue` container adaptors, like many other components of the C++ Standard Library, were inspired by SGI's Standard Template Library (STL). The STL documentation [STL] explains why separate `top`/`front` and `pop` methods are provided: `pop` cannot return by reference since the reference would dangle, but returning by value would impose on the user the cost of copying the object. Because imposing this cost would be undesirable, I do not propose to change the existing `pop` methods to return the popped value, but instead propose a new `pop_value` method. In modern C++, the cost of calling `pop_value` will typically be minimal due to move semantics. For example, here is a possible implementation of `std::stack::pop_value`:

```
value_type pop_value() {
    value_type x = move(c.back());
    c.pop();
    return x;
}
```

In the above implementation, NRVO allows `x` to be treated as an alias for the returned object (§11.9.6 [class.copy.elision][1]p1.1 of the Standard). GCC always performs this optimization by default and Clang performs it at `-O1` and higher (but both permit the user to explicitly disable it using the `-fno-elide-constructors` flag). When NRVO is performed, a single call to `value_type`'s move constructor occurs when `pop_value` is called. When NRVO is not performed, the move constructor is called twice (§7.5.4.2 [expr.prim.id.unqual]).

§ 3.2 Exception safety

The more well-known reason for `pop` not returning the popped value is the issue of exception safety. In the implementation of `pop_value` above, although NRVO will often be performed, it is not guaranteed. Now suppose that NRVO is not performed. Then, the returned object is initialized by moving from `x`. If this move operation exits via an exception, it is impossible to provide the strong exception safety guarantee. In order to restore the state of the stack or queue as it existed prior to the call to `pop_value`, the popped object would need to be put back at the top of the stack or the front of the queue. This operation cannot be accomplished without calling the move constructor again (*i.e.*, the very operation that has just failed). Therefore, the only choice is to destroy the popped object and propagate the exception. The object popped is thus "dropped" and its value is lost forever.

§ 4 Solution

§ 4.1 `std::stack` and `std::queue`

It is possible to implement `std::stack<T>::pop_value` and `std::queue<T>::pop_value` so that they provide the same exception safety guarantee as calling `top` or `front` first and then calling `pop`, and, therefore, do not drop elements when `T`'s move constructor throws. The following is a proof-of-concept implementation for `std::stack`. The implementation for `std::queue` is very similar.

```
value_type pop_value() {
    struct Guard {
        stack* __this;
        bool failure = false;
        ~Guard() noexcept(false) {
            if (!failure) {
                __this->c.pop_back();
            }
        }
    } guard{this};
    try {
        return move(c.back());
    } catch (...) {
        guard.failure = true;
        throw;
```

```
    }
}
```

In this implementation, only a single move occurs, even if the compiler declines to perform NRVO. Thus, there is only a single point where we must pay attention to the possibility of a move constructor exiting by throwing an exception. Note that it is the caller's responsibility to ensure that the popped value is not lost *after* `pop_value` returns. For example:

```
void foo(std::stack<Widget>& s) {
    Widget w = s.pop_value();
    // do something with `w` ...
    w = s.pop_value();
}
```

In the above code, due to guaranteed copy elision [P0135R1], the object stored in the underlying container `c` is moved directly into the local variable `w` when `w` is initialized. If this move operation succeeds, the moved-from object at the top of the stack is popped from `c`. If the move operation fails by throwing an exception, a flag is set and the destruction of `guard` does not attempt to remove the object from the underlying container. Therefore, if `value_type`'s move constructor provides the strong exception safety guarantee, then the `pop_value` implementation above provides the same guarantee. If `value_type`'s move constructor provides the basic exception safety guarantee, then `pop_value` provides the basic exception safety guarantee in that the stack remains in a valid state, and additionally guarantees that the item at the top of the stack is not lost (though its original value might not be recoverable).

However, if the move assignment operator of `Widget` throws after the second call to `pop_value`, then, even if that operator provides the strong exception safety guarantee, the temporary object materialized from `s.pop_value()`, which still holds the popped value, will be destroyed and the value will be lost. If the caller of `pop_value` is able to recover from a move operation that throws, then the caller can avoid dropping the popped value by declaring another local variable to hold the popped value before calling the move assignment operator. Of course, such concerns need not trouble a user who has provided `Widget` with a non-throwing move assignment operator.

Note that the above implementation is simply a proof-of-concept; one optimization that I anticipate library implementors wanting to make (but that is not necessary to specify in the Standard) is that if `std::is_nothrow_move_constructible_v<value_type>` is true, then the try/catch block and the `failure` variable can be omitted.

§ 4.1.1 Throwing `pop_back` operations

The above analysis assumes that the underlying `pop_back` or `pop_front` method never fails by throwing an exception. It is unusual for such an exception to even be possible[2], but in rare cases in which such an exception is actually thrown, it is not possible for `pop_value` to successfully return unless the exception is swallowed, because propagating the exception will destroy the already-constructed return object (§14.3 [except.ctor]p2).

Therefore, the popped value would be lost. To prevent this outcome, one possibility is to provide the `pop_value` method only when it is known that `pop_back` or `pop_front` will never throw an exception; however, since these methods might not have a `noexcept` specification, such a restriction would significantly reduce the utility of the proposed `pop_value` method. Instead, I propose to simply let the exception propagate. This rare situation, in which the user has chosen an unusual underlying container, is the only one in which the use of `pop_value` can result in an element being dropped. In some cases, the caller may be able to tell whether an element was dropped, *i.e.*, when the exception thrown by `pop_value` is one that cannot be thrown by the move constructor.

Two possible alternatives (not currently proposed) offer the user the ability to avoid this situation or to detect and respond to it more reliably:

1. During the execution of `pop_value`, `std::terminate` is called if `pop_back` or `pop_front` exits by throwing an exception; therefore, the calling code can assume that this never occurs. (In the above implementation, such termination can be made to occur automatically by removing the `noexcept` specification on `Guard`'s destructor.)

2. a. During the execution of `pop_value`, if `pop_back` or `pop_front` exits by throwing an exception, then `pop_value` catches the exception and wraps it in a new exception type called `std::pop_failed` that is derived from `std::nested_exception`. By catching `std::pop_failed`, the caller can check whether the exception came from the move constructor or from the underlying container.[3]

   b. Same as option (a), but the specifications of the existing `pop` methods of `std::stack` and `std::queue` are also changed to throw `std::pop_failed`.

These alternatives are not currently proposed because the termination imposed by alternative 1 may be undesirable for some users, while alternative 2 adds complexity to the proposal without a known use case.

§ 4.1.2 `move` versus `move_if_noexcept`

In cases where `value_type`'s move constructor is not `noexcept`, there is a question of whether the implementation of `pop_value` should call `std::move` or `std::move_if_noexcept`.

The most well-known usage of `std::move_if_noexcept` is by `std::vector` when reallocating. Copying, rather than moving, objects with throwing move constructors during reallocation enables `std::vector` to provide the strong exception safety guarantee for some operations that may reallocate.

For the proposed `pop_value` method of `std::stack` and `std::queue`, it is also possible to provide the strong exception guarantee unconditionally (*i.e.*, regardless of what `value_type` is) by using `std::move_if_noexcept` instead of `std::move`. However, using

`std::move_if_noexcept` also has a cost because it results in possibly unexpected and expensive copies for reasons that are unfamiliar even to many experienced C++ programmers (for example, if the value type is `std::deque<T>` with some implementations). In the case of `pop_value`, I believe there are two differences from vector reallocation that tip the balance in favor of using `std::move` and not `std::move_if_noexcept`:

1. `pop_value` uses only a *single* call to the move constructor; therefore, if `pop_value` uses `std::move`, it inherits the exception safety guarantee from `T`'s move constructor: in other words, if `T`'s move constructor provides the strong exception safety guarantee, then so does `pop_value`. The same is not true for vector reallocation.
2. In cases where the user needs strong exception safety unconditionally (*i.e.*, even when `T`'s move constructor is not known to provide the strong exception safety guarantee), the user can fall back on calling `top`, calling `std::move_if_noexcept` themselves, then calling `pop`. In contrast, if vector reallocation were to use `std::move`, it would be much more inconvenient for the user to recover the strong exception safety guarantee. [4]

§ 4.2 `std::priority_queue`

The case of `std::priority_queue` is very different. It is not possible to move from the top of a heap without compromising the heap's invariant, so `std::pop_heap` must be called on the underlying container before the returned object can be initialized. In general, `std::pop_heap` will use a number of move operations that is approximately logarithmic in the current size of the heap, and if any such operation besides the first throws an exception, the heap will be left in an invalid state. This risk also exists in the status quo: although the user will call `top` first and `pop` afterward, a failure in `std::pop_heap` during the second step will make it impossible to extract any further data safely from the `priority_queue`. Therefore, `std::priority_queue` is inherently unreliable in the presence of throwing moves. For this reason, I think that the possibility of throwing moves is no rationale for omitting `pop_value` from `std::priority_queue`. Furthermore, if a move operation were to throw, then it is more likely for the exception to occur during the `pop_heap` step than the initialization of the return object. Therefore, I consider it unnecessary to provide a guarantee about the state of the `priority_queue` even if the final move throws; the caller cannot tell whether the exception originated from the `pop_heap` step, and therefore, cannot rely on any such guarantee. For example, a conforming implementation need not check whether the final move succeeded:

```
value_type pop_value() {
    pop_heap(c.begin(), c.end(), comp);
    struct Guard {
        priority_queue* __this;
        ~Guard() noexcept(false) {
            __this->c.pop_back();
        }
```

```
    } guard{this};
    return move(c.back());
}
```

§ 5 Alternative names

The name `pop_value` was suggested by Lauri Vasama. Other possible names include `pop_and_get` and `pop_and_return`.

§ 6 Wording

The proposed wording is relative to [N4971].

Modify §9.4.5 [dcl.init.list]p1:

> […] List-initialization can occur in direct-initialization or copy-initialization contexts; list-initialization in a direct-initialization context is called *direct-list-initialization* and list-initialization in a copy-initialization context is called *copy-list-initialization*. Direct-initialization that is not list-initialization is called *direct-non-list-initialization*; copy-initialization that is not list-initialization is called *copy-non-list-initialization*.
> […]

Modify §24.6.6.1 [queue.defn]p1:

```
// ...
void pop()                              { c.pop_front(); }
value_type pop_value();
void swap(queue& q) noexcept(...)
// ...
```

Add a new paragraph to the end of §24.6.6.4 [queue.mod]:

```
value_type pop_value();
```

Effects: Copy-non-list-initializes the return object from `std::move(c.front())`, then calls `c.pop_front()`.

Modify §24.6.7.1 [priqueue.overview]p1:

```
// ...
void pop();
value_type pop_value();
void swap(priority_queue& q) noexcept(...)
// ...
```

Add a new paragraph to the end of §24.6.7.4 [priqueue.members]:

```
value_type pop_value();
```

Effects: Calls `pop_heap(c.begin(), c.end(), comp)`, then copy-non-list-initializes the return object from `std::move(c.back())`, then calls `c.pop_back()`.
Remarks: If this function exits by throwing an exception, the value of `c` is unspecified and is possibly not a valid heap ([alg.heap.operations.general]).

Modify §24.6.8.2 [stack.defn]p1:

```
// ...
void pop()                          { c.pop_back(); }
value_type pop_value();
void swap(stack& s) noexcept(...)
// ...
```

Add a new paragraph to the end of §24.6.8.5 [stack.mod]:

```
value_type pop_value();
```

Effects: Copy-non-list-initializes the return object from `std::move(c.back())`, then calls `c.pop_back()`.

§ 7 References

[N4971] Thomas Köppe. 2023-12-18. Working Draft, Programming Languages — C++.
https://wg21.link/n4971

[P0135R1] Richard Smith. 2016-06-20. Wording for guaranteed copy elision through simplified value categories.
https://wg21.link/p0135r1

[STL]
http://web.archive.org/web/20010221200527/http://www.sgi.com/tech/stl/stack.html

---

1. All citations to the Standard are to working draft N4971 unless otherwise specified. ↩

2. The `pop_front` and `pop_back` methods of `std::vector`, `std::deque`, and `std::list` never throw an exception (§24.2.2.2 [container.reqmts]p66.3). ↩

3. If `value_type`'s move constructor happens to propagate an exception from a failed pop operation that it invokes, the caller needs some way to tell that the exception thrown by `pop_value` came from the move constructor and not from the direct call to `pop_front` or `pop_back`. Therefore, the `pop_failed` exception class would also need to hold a "depth" counter: when `pop_value` catches an exception of type `pop_failed` thrown by the move constructor, it would need to increment that counter before rethrowing, whereas a `pop_failed` that comes from the direct call to `pop_front` or `pop_back` would be initialized with a depth of 0. ↩

4. The technique for doing so would be: suppose one wishes to add an element to the end of the vector. If `T` is nothrow-move-constructible, or if `size() < capacity()`,

then just call `push_back` normally. Otherwise, create a new empty vector, reserve some multiple of the original vector's capacity, copy-insert the old vector's elements into it followed by the new element, swap the new vector with the old one, and then destroy the new one. ↩